

PYCON CHINA 2026

是时候学点 Rust 🦀 了

从 `pip install` 变快说起

Lightning Talk

2026-9-5

Rust 就在你身边

命令行工具

astral-sh/**uv**

An extremely fast Python package and project manager, written in Rust.



👤 602 Contributors
🛠️ 9 Used by
★ 89k Stars
🍴 4k Forks



facebook/**pyrefly**

A fast type checker and language server for Python



👤 259 Contributors
🕒 509 Issues
💬 39 Discussions
★ 7k Stars
🍴 498 Forks

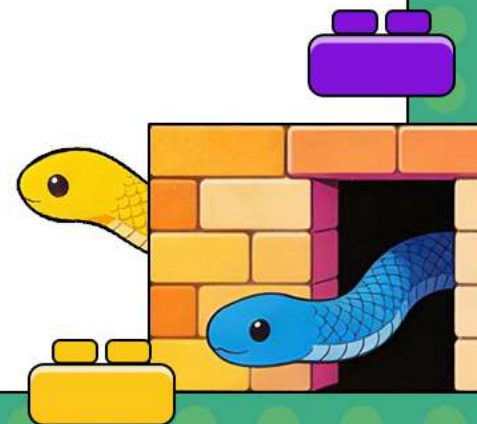


astral-sh/**ruff**

An extremely fast Python linter and code formatter, written in Rust.



👤 940 Contributors
🛠️ 58 Used by
💬 461 Discussions
★ 49k Stars
🍴 2k Forks



Rust 就在你身边 continued

第三方库

pydantic/pydantic

Data validation using Python type hints



791 Contributors 980k Used by 2k Discussions 29k Stars 3k Forks

pyca/cryptography

cryptography is a package designed to expose cryptographic primitives and recipes to Python developers.



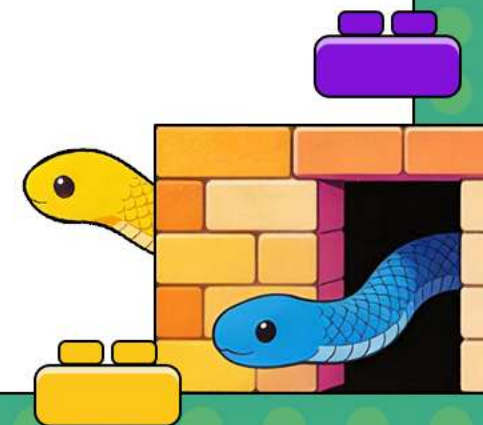
368 Contributors 968k Used by 8k Stars 2k Forks

pola-rs/polars

Extremely fast Query Engine for DataFrames, written in Rust



742 Contributors 7 Used by 40k Stars 3k Forks



CPython 源码或将迎来 Rust

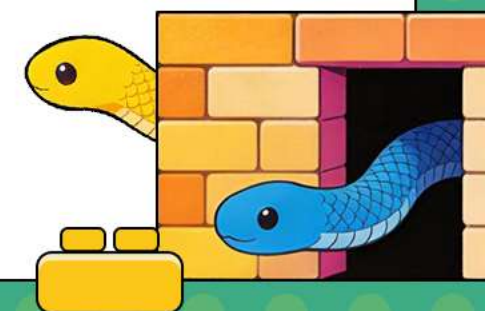
- 2025-11 Pre-PEP: Rust for CPython 讨论帖
- 2026-3 CI 全平台构建 
- 2026-5 PyCon US zlib-py (zlib-rs)
- 2026-7 PEP 草案定稿，提交讨论（推迟）
- 2026-8 内部 Rust API 设计文档（评审中）
- 2026-10 Core Dev Sprints (Subinterpreter support)
- 2027-5 Python 3.16 beta 1



github.com/Rust-for-CPython/cpython

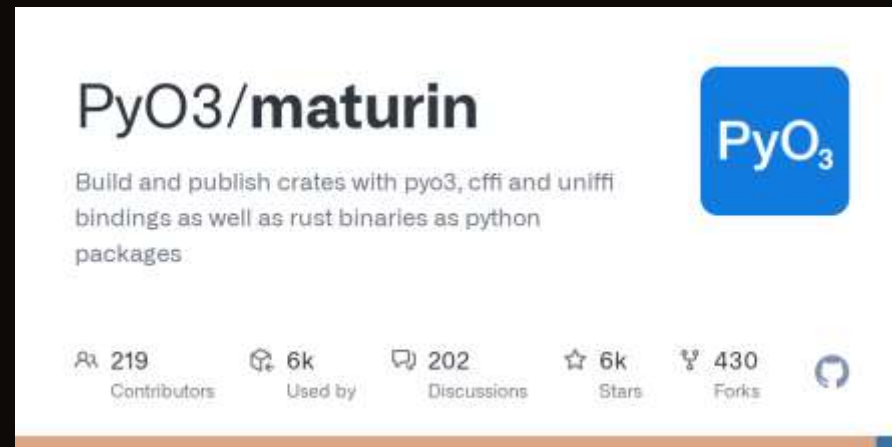
PEP 较原计划顺延 —— 不是搁浅，是 API 设计细节值得慢慢推敲

Operation	Size	CPython	zlib	zlib_py	Speedup
decompress	1 MB (L6)		464 μs	91 μs	5.1× faster
decompress	10 MB (L6)		5.24 ms	868 μs	6.0× faster
stream compress	1 MB (L6)		1.52 ms	292 μs	5.2× faster
stream compress	10 MB (L6)		15.42 ms	2.77 ms	5.6× faster
stream decompress	1 MB (L6)		469 μs	127 μs	3.7× faster
compress	10 MB (L9)		14.82 ms	8.12 ms	1.8× faster
adler32	1 MB		273 μs	28 μs	9.7× faster
adler32	10 MB		2.77 ms	281 μs	9.9× faster
crc32	1 MB		34 μs	99 μs	2.9× slower
crc32	10 MB		343 μs	1.00 ms	2.9× slower
compress binary	1 MB (L6)		14.63 ms	12.03 ms	1.2× faster

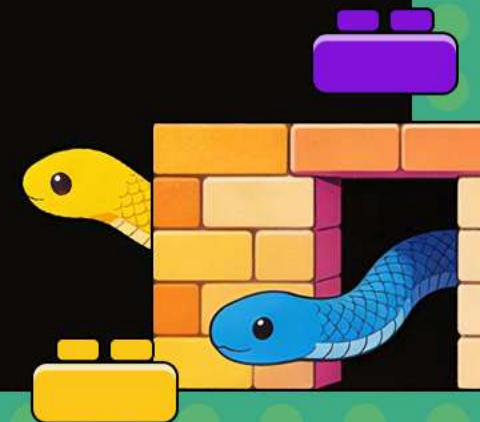


把大象装进冰箱：三步实现 Python 调用 Rust

第一步



```
terminal  
  
uv tool install maturin  
maturin init --name fibonacci_rs --bindings pyo3
```



把大象装进冰箱：三步实现 Python 调用 Rust

第二步

```
src/lib.rs

use pyo3::prelude::*;

#[pymodule]
mod fibonacci_rs {
    use pyo3::prelude::*;

    /// Calculate nth fibonacci number using iteration
    #[pyfunction]
    fn fibonacci(n: u64) -> u64 {
        let mut a: u64 = 0;
        let mut b: u64 = 1;

        for _ in 0..n {
            let temp: u64 = b;
            b = a.wrapping_add(b);
            a = temp;
        }

        a
    }
}
```



把大象装进冰箱：三步实现 Python 调用 Rust

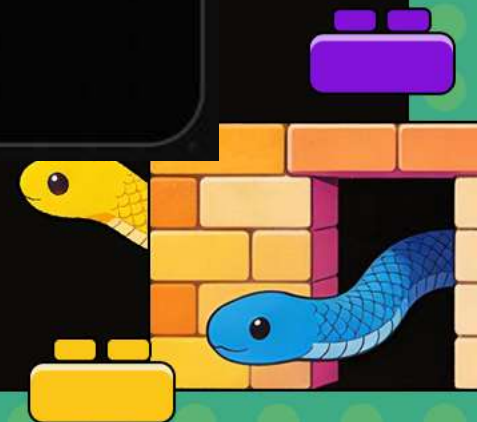
第三步

```
terminal

maturin develop
uv run python -c "import fibonacci_rs;print(fibonacci_rs.fibonacci(88))"

1100087778366101931
```

Python 负责编排, Rust 负责计算



还能再给力一点吗?

Julia set

把复平面上每个点反复平方再加一个常数 c , 看哪些点不会飞向无穷

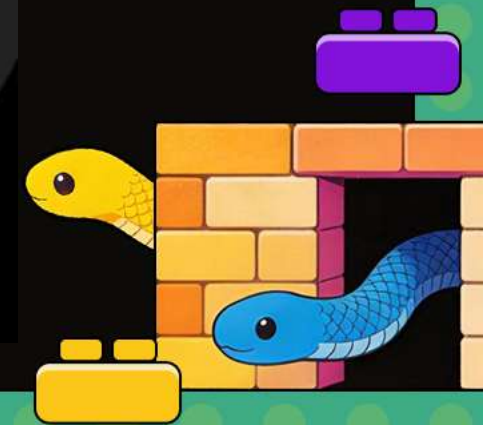
`height = width = 1,000`

`z = z * z + c`

`c = -0.62772 - 0.42193 i`

`N = 300`

`| z | < 2 ?`



还能再给力一点吗? continued

```
julia_pure.py

def calculate_z_serial_purepython(
    maxiter: int, zs: list[complex], cs: list[complex]
) -> list[int]:
    """Calculate output list using Julia update rule"""

    output: list[int] = [0] * len(zs)

    for idx, z in enumerate(zs):
        n = 0
        c: complex = cs[idx]

        while n < maxiter and abs(z) < 2:
            z: complex = z * z + c
            n += 1

        output[idx] = n

    return output
```

```
terminal

uv pip install line_profiler
kernprof -l -v julia_pure.py
```

Per Hit	% Time	Line Contents
=====		
		@profile
		def calc_pure_python(desired_width: float, max_iterations: int) -> None:
		# build a list of coordinates and the initial condition for each cell.
0.2	0.0	zs: list[complex] = []
0.2	0.0	cs: list[complex] = []
0.2	0.0	for ycoord in y:
0.2	0.9	for xcoord in x:
0.2	1.4	zs.append(complex(xcoord, ycoord))
0.3	1.5	cs.append(complex(c_real, c_imag))
1.9	0.0	start_time: float = time.perf_counter()
64		
65	1	16295284.3 1.63e+07 96.1
66		
67	1	2.1 2.1 0.0
		end_time: float = time.perf_counter()

还能再给力一点吗? continued

```
src/lib.rs

#[pymodule]
mod julia_set_rs {
    use pyo3::prelude::*;
    use pyo3::types::{PyComplex, PyList};
    ...
    #[pyfunction]
    fn calculate_z_serial(
        maxiter: u32,
        zs: &Bound<'_, PyList>,
        cs: &Bound<'_, PyList>,
    ) -> PyResult<Vec<u32>> {
        ...

        for (idx, z) in zs.iter().enumerate() {
            let mut n: u32 = 0;
            let (cre, cimg): (f64, f64) = cs[idx];
            let (mut zre, mut zim): (f64, f64) = *z;

            while n < maxiter && zre * zre + zim * zim < 4.0 {
                // (re + i*im)^2 + c
                let new_re: f64 = zre * zre - zim * zim + cre;
                zim = 2.0 * zre * zim + cimg;
                zre = new_re;
                n += 1;
            }

            output[idx] = n;
        }

        Ok(output)
    }
}
```

```
julia_with_rs.py

from julia_set_rs import calculate_z_serial
```

```
...
```

```
def calc_with_rs(desired_width: float, max_iterations: int) -> None:
```

```
...
```

```
    start_time: float = time.perf_counter()
```

```
    output: list[int] = calculate_z_serial(max_iterations, zs, cs)
```

```
    end_time: float = time.perf_counter()
```

```
...
```

还能再给力一点吗? continued

Line #	Hits	Time	Per Hit	% Time	Line Contents
11					@profile
12					def calc_with_rs(desired_width: float, max_iterations: int) -> None:
31					
32					# build a list of coordinates and the initial condition for each cell.
33	1	0.2	0.2	0.0	zs: list[complex] = []
34	1	0.2	0.2	0.0	cs: list[complex] = []
35					
36	1001	160.8	0.2	0.0	for ycoord in y:
37	1001000	156383.4	0.2	19.3	for xcoord in x:
38	1000000	264274.1	0.3	32.7	zs.append(complex(xcoord, ycoord))
39	1000000	238166.4	0.2	29.5	cs.append(complex(c_real, c_imag))
44	1	2.3	2.3	0.0	start_time: float = time.perf_counter()
45					
46	1	145207.1	145207.1	18.0	output: list[int] = calculate_z_serial(max_iterations, zs, cs)
47					
48	1	4.1	4.1	0.0	end_time: float = time.perf_counter()

时间对比 3.008 sec -> 0.144 sec



思维模式

推荐显式 (Explicit) -> 强制显式

- Zen of Python (PEP-20) "显式优于隐式"只是建议, 类型标注可写可不写
- Rust: mut 要声明、match 要穷尽, 不照办直接拒绝编译

引用数据 -> 拥有/借用数据

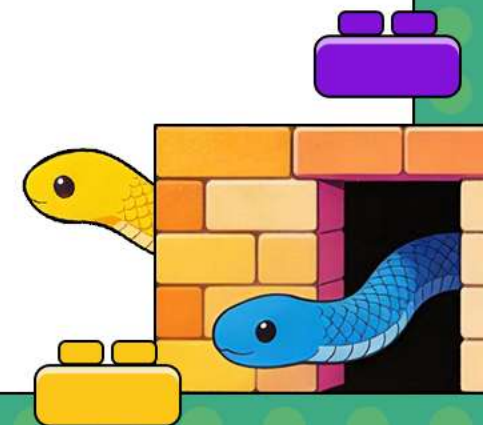
- Python: 赋值只是贴名字标签, 随便共享, GC 兜底回收
- Rust: 每个值有唯一 owner, move/借用规则编译期检查

EAFP (Easier to Ask Forgiveness than Permission) -> LBYL (Look Before You Leap)

- Python: 先 try 再 except, "宽恕比许可更容易" 是惯用法
- Rust: 会不会失败写进类型里, Option/Result 不检查就没法用

GIL -> 真并发

- Python: CPU 密集? multiprocessing / free-threaded build
- Rust: 数据竞争 (Data Race) 是编译错误, "无畏并发"



你可以参考的入门路线图

第 1 阶段：语法桥接

- 基本类型、容器类型、控制流、函数、struct、枚举

第 2 阶段：爬坡段（也许是最痛苦的阶段）

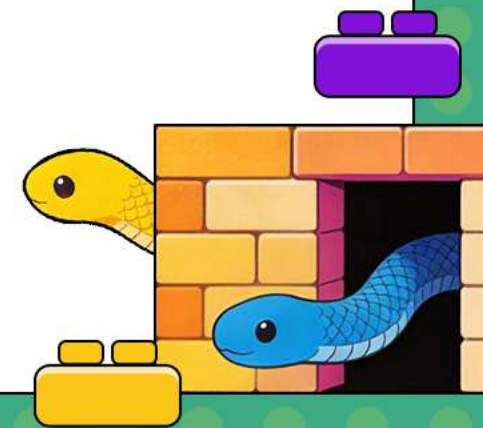
- 所有权与借用
- 错误处理
- Traits 与泛型
- 闭包与迭代器

第 3 阶段：并发

- std::thread, Send/Sync
- Arc / Mutex
- Rayon, tokio

第 4 阶段：动手实践

- PyO3 + maturin 改写自己项目里的一个热点函数
- profile 定位 -> parity 测试 -> benchmark 对比



Get your hands "dirty"

Rustlings

Current	State	Name	Path
	DONE	intro1	exercises/00_intro/intro1.rs
	DONE	intro2	exercises/00_intro/intro2.rs
	DONE	variables1	exercises/01_variables/variables1.rs
	DONE	variables2	exercises/01_variables/variables2.rs
	DONE	variables3	exercises/01_variables/variables3.rs
	DONE	variables4	exercises/01_variables/variables4.rs
	DONE	variables5	exercises/01_variables/variables5.rs
	DONE	variables6	exercises/01_variables/variables6.rs
	DONE	functions1	exercises/02_functions/functions1.rs
	DONE	functions2	exercises/02_functions/functions2.rs
	DONE	functions3	exercises/02_functions/functions3.rs
	DONE	functions4	exercises/02_functions/functions4.rs
	DONE	functions5	exercises/02_functions/functions5.rs
	DONE	if1	exercises/03_if/if1.rs
	DONE	if2	exercises/03_if/if2.rs
	DONE	if3	exercises/03_if/if3.rs
	DONE	quiz1	exercises/quizzes/quiz1.rs
	DONE	primitive_types1	exercises/04_primitive_types/primitive_types1.rs
	DONE	primitive_types2	exercises/04_primitive_types/primitive_types2.rs
	DONE	primitive_types3	exercises/04_primitive_types/primitive_types3.rs
	DONE	primitive_types4	exercises/04_primitive_types/primitive_types4.rs
	DONE	primitive_types5	exercises/04_primitive_types/primitive_types5.rs
	DONE	primitive_types6	exercises/04_primitive_types/primitive_types6.rs
W >>>>>>	PENDING	vecs1	exercises/05_vecs/vecs1.rs
	PENDING	vecs2	exercises/05_vecs/vecs2.rs
	PENDING	move_semantics1	exercises/06_move_semantics/move_semantics1.rs

CodeWars

8 kyu Beginner - Lost Without a Map ✓
 ☆ 1195 🏆 322 📈 91% of 11,995 💰 2,859 of 221,650 👤 PG1
 ⚠ 3 Issues Reported

Instructions Output Past Solutions

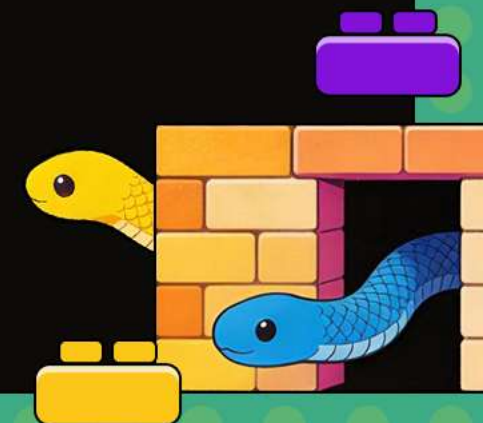
Given an array of integers, return a new array with each value doubled.

For example:

[1, 2, 3] --> [2, 4, 6]

FUNDAMENTALS ARRAYS

LeetCode



什么时候值得写点 Rust?

profiling 先行 (确认性能瓶颈)

可以考虑:

- CPU 密集
- 边界清晰
- 处理不可信输入

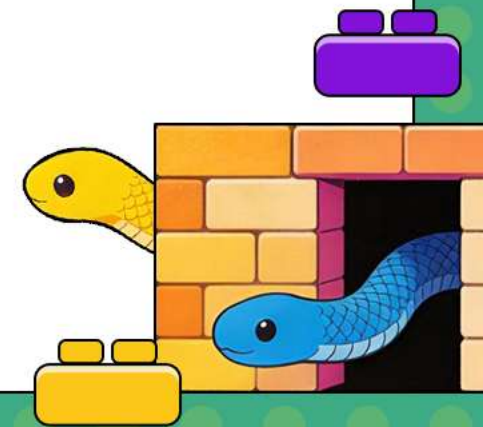
替代方案?

- 优化算法
- free-threaded (no-GIL)
- 现成的第三方库
- Cython / numba / PyPy
- GPU 加速

可以不改:

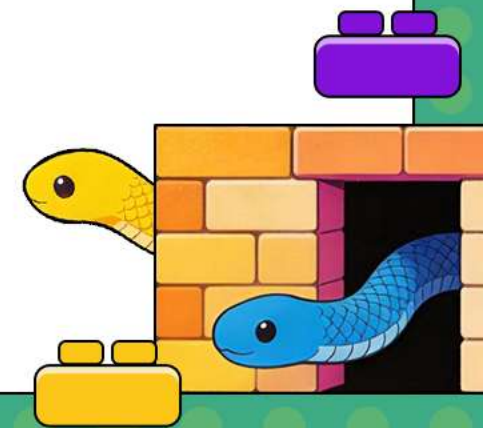
- I/O 密集
- 胶水代码

Rust ≠ 无脑快, 快的从来是选对了场景。

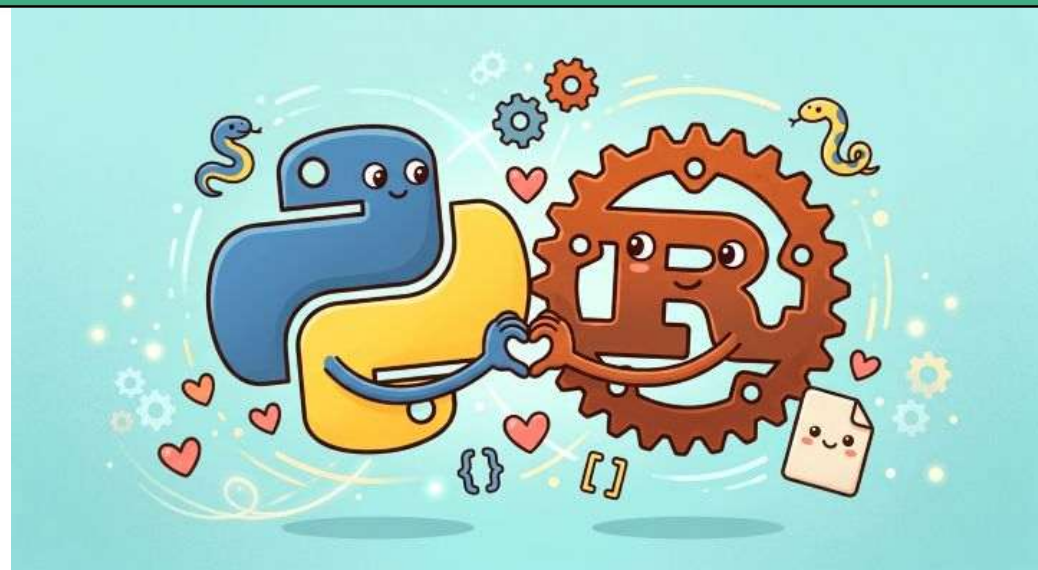


Rust 还是太难学了?

即使永远不写生产级 Rust, 显式类型、所有权、显式错误处理、穷尽匹配 等等思维习惯也会迁移回来, 让你写出更严谨的 Python



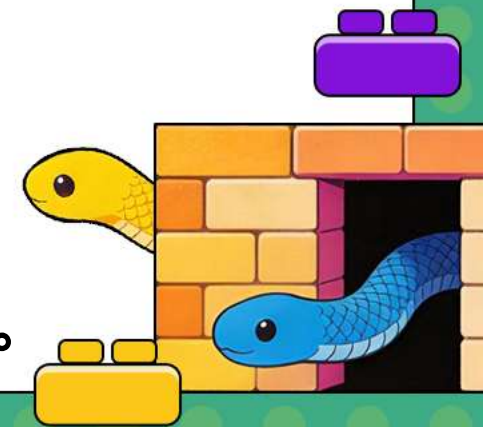
结语 Let's wrap it up!



Rust 不是 Python 的对手，而是它的伙伴。

学 Rust 不只是个人技能投资，也可能是未来参与 CPython 的入场券。

也许下一个为 Rust for CPython 做贡献的就是你。



Credits

Microsoft - <Rust for Python Programmers>

Daroc Alden - "Eventual Rust in Cpython"

Bob Belderbos - "Learning Rust Made Me a Better Python Developer"

Daniel Szoke - "Why You, a Python Developer, Should Learn Rust"

Emma Smith - "Rust for CPython: Making Python Safer and More Robust for Everyone"

Micha Gorelick, Ian Ozsvald - <High Performance Python, 2nd Edition>



感谢聆听! Thank You for Your Time!

演示代码: github.com/linzeyang/pycon-china-2026

About Me (A Pythonista for over a decade)



linzeyang



@linzeyang.bsky.social



@zeyanglin@mastodon.online

小红书

蓝莓派



蓝莓派



BlueberryPy

